Advanced Kafka Operations

Day 4 — Exclusive Level 3 Support Topics

Rajesh Pasham | Kafka Corporate Training

Day 4 — Agenda

01

Kafka Internals

Replica Manager · Group Coordinator · Controller

02

Performance Tuning

Broker · Producer · Consumer · OS Tuning

03

Cluster Scaling

Adding Brokers · Rebalancing · Kafka Connect · Schema Registry

04

Security

TLS/mTLS · Kerberos · SASL (PLAIN, SCRAM)

MODULE 01

Kafka Internals

Replica Manager — Overview

- Core broker-side component managing all partition replicas
- Handles ProduceRequests: appends messages to the leader's log
- Handles FetchRequests: from follower replicas and consumers
- Maintains and updates the ISR (In-Sync Replicas) list
- Triggers ISR shrink when a replica falls behind
- Triggers ISR expand when follower catches back up
- Key metric: UnderReplicatedPartitions (should always be 0 in healthy cluster)
- Configured via: num.replica.fetchers, replica.lag.time.max.ms

In-Sync Replicas (ISR) Deep Dive

What is ISR?

- Set of replicas fully caught up with leader
- Only ISR replicas can become the new leader
- acks=all waits for all ISR replicas to ack
- Tracked per-partition on the leader
- Shrinks when follower falls behind (lag > threshold)
- Expands when follower catches up

Key Config

- replica.lag.time.max.ms=30000
- → Remove from ISR after 30s of lag
- replica.fetch.max.bytes=1048576
- → Max bytes per fetch request
- num.replica.fetchers=4
- → Parallel replication threads
- min.insync.replicas: min ISR for writes

Group Coordinator — Consumer Group Management

- Each consumer group is assigned to exactly ONE Group Coordinator broker
- Consumers send JoinGroup requests → Coordinator assigns partitions
- One consumer is elected Group Leader (manages partition assignment)
- Coordinator tracks committed offsets in __consumer_offsets topic
- Manages heartbeat timeouts (session.timeout.ms)
- Two rebalance protocols: Eager (stop-the-world) and Cooperative (incremental)
- CooperativeStickyAssignor minimizes partition movements during rebalance
- Consumer group states: Empty → PreparingRebalance → Stable → Dead

The Kafka Controller — Cluster Brain

Who is Controller?

One broker is elected Controller via ZooKeeper (/controller node) or KRaft Raft quorum. Only one at any time.

Responsibilities

Elects partition leaders on broker failure, manages partition/replica states, handles topic create/delete.

KRaft Mode (Kafka 3.3+)

ZooKeeper-free. Controllers form a Raft quorum. Metadata stored in __cluster_metadata topic. Faster & simpler.

Key Metrics

ActiveControllerCount (must be 1), OfflinePartitionsCount (must be 0), LeaderElectionRateAndTimeMs

MODULE 02

Performance Tuning

Broker Performance Tuning

I/O & Disk

- Use SSDs / NVMe for log directories
- XFS filesystem for large sequential writes
- Mount with noatime option
- JBOD for better throughput distribution
- num.io.threads=16 (disk I/O threads)
- num.network.threads=6 (network I/O threads)

Log & Memory

- log.segment.bytes=1073741824 (1GB segments)
- log.retention.hours=168 (7 day retention)
- log.cleaner.threads=2 (compaction threads)
- JVM Heap: -Xmx6g -Xms6g (fixed size)
- Use G1GC: -XX:+UseG1GC
- MaxGCPauseMillis=20 for low latency

Producer Performance Tuning

Key configs to maximize throughput with reliability:

```
# Throughput settings
batch.size=131072              # 128KB batches (default 16KB)
linger.ms=10                  # Wait 10ms to fill batches
compression.type=snappy       # CPU-efficient compression
buffer.memory=67108864        # 64MB producer buffer

# Reliability settings
acks=all                      # Wait for all ISR replicas
enable.idempotence=true       # Exactly-once delivery
retries=2147483647            # Retry on failures
max.in.flight.requests.per.connection=5
```

Consumer Performance Tuning

Tune fetch parameters for high-throughput consumption:

```
# Fetch optimization
fetch.min.bytes=1048576      # Wait for 1MB before returning
fetch.max.bytes=52428800     # 50MB max per fetch
fetch.max.wait.ms=500        # Max wait time for data
max.partition.fetch.bytes=1048576

# Poll tuning
max.poll.records=1000        # Messages per poll()
max.poll.interval.ms=300000  # Time between polls

# Group rebalance
session.timeout.ms=45000
heartbeat.interval.ms=3000   # 1/3 of session.timeout.ms
```

OS-Level Tuning for Kafka Brokers

Critical Linux kernel and system settings:

```
# Reduce swappiness — Kafka needs OS page cache
echo 1 > /proc/sys/vm/swappiness

# File descriptor limits
ulimit -n 1000000
# /etc/security/limits.conf:
kafka  soft  nofile  1000000
kafka  hard  nofile  1000000

# Network buffers
sysctl -w net.core.wmem_max=134217728
sysctl -w net.core.rmem_max=134217728

# Dirty page flushing
sysctl -w vm.dirty_background_ratio=5
```

MODULE 03

Cluster Scaling

Adding Brokers — Step by Step

- Step 1: Configure new broker with unique broker.id and correct ZooKeeper/KRaft settings
- Step 2: Start the broker — it joins the cluster automatically
- Step 3: Verify broker membership (kafka-broker-api-versions.sh)
- Step 4: New broker receives NO existing partitions automatically — must trigger reassignment
- Step 5: Generate reassignment plan using kafka-reassign-partitions.sh --generate
- Step 6: Execute plan: --execute (throttle with leader.replication.throttled.rate for safety)
- Step 7: Monitor progress: --verify
- Step 8: Remove throttle configs after reassignment completes

Kafka Connect Deep Dive

Workers

JVM processes hosting connectors and tasks. In distributed mode, workers form a cluster with fault tolerance.

Connectors & Tasks

Connector = high-level config. Tasks = units of parallelism doing actual data movement. More tasks = more parallelism.

Standalone vs Distributed

Standalone: single worker, file config, dev/test only. Distributed: REST API, fault-tolerant, use in production.

SMTs & Converters

SMTs: inline message transforms (mask fields, route, timestamp). Converters: serialize between Connect and Kafka (Avro, JSON, String).

Schema Registry Deep Dive

What It Does

- Centralized schema store for Avro/JSON/Protobuf
- Schema ID embedded in every message (5 bytes)
- Ensures consumer can always deserialize messages
- Enforces schema compatibility rules
- REST API for schema management
- Works with Kafka Connect, Kafka Streams, apps

Compatibility Modes

- BACKWARD: New schema reads old data
- → Safe for consumer upgrades first
- FORWARD: Old schema reads new data
- → Safe for producer upgrades first
- FULL: Both BACKWARD + FORWARD
- NONE: No compatibility check
- TRANSITIVE: Checks ALL prior versions

MODULE 04

Kafka Security

TLS & mTLS — Encryption in Transit

TLS Setup (Broker)

- listeners=SSL://localhost:9093
- ssl.keystore.location=kafka.server.keystore.jks
- ssl.keystore.password=<password>
- ssl.truststore.location=kafka.server.truststore.jks
- ssl.truststore.password=<password>
- TLS encrypts all network traffic
- Use OpenSSL + keytool for cert management

mTLS (Client Auth)

- ssl.client.auth=required
- → Broker demands client certificate
- Client also provides keystore:
- ssl.keystore.location=client.keystore.jks
- ssl.keystore.password=<password>
- mTLS = mutual authentication
- Best practice: automate cert rotation

Kerberos (SASL/GSSAPI) — Enterprise Auth

Configuration for Kafka broker with Kerberos:

```
# server.properties
listeners=SASL_PLAINTEXT://localhost:9092
security.inter.broker.protocol=SASL_PLAINTEXT
sasl.mechanism.inter.broker.protocol=GSSAPI
sasl.enabled.mechanisms=GSSAPI
sasl.kerberos.service.name=kafka

# kafka_server_jaas.conf
KafkaServer {
  com.sun.security.auth.module.Krb5LoginModule required
  useKeyTab=true storeKey=true
  keyTab="/etc/kafka/kafka.keytab"
  principal="kafka/broker1.example.com@REALM";
};
```

SASL Mechanisms — PLAIN vs SCRAM

SASL/PLAIN

- Username + password authentication
- Credentials in JAAS config file
- Cleartext — ALWAYS pair with TLS
- Simple to configure, good for dev/test
- users defined in server JAAS config
- sasl.enabled.mechanisms=PLAIN
- Not recommended for high-security prod

SASL/SCRAM-SHA-512

- Salted challenge-response protocol
- Credentials stored as salted hashes
- Stronger than PLAIN — safe without TLS*
- Supports SHA-256 and SHA-512
- Credentials managed via kafka-configs.sh
- sasl.enabled.mechanisms=SCRAM-SHA-512
- Recommended for production environments

Production Security — Recommended Stack

🔐 Encryption (TLS)

TLS on all listeners. Generate certs with openssl + keytool. Automate rotation with cert-manager or Vault.

🪪 Authentication

SCRAM-SHA-512 for applications. GSSAPI/Kerberos in enterprise AD/LDAP environments. mTLS for service-to-service.

🛡 Authorization (ACLs)

Use AclAuthorizer. Grant topic-level READ/WRITE/DESCRIBE per principal. Audit regularly. Principle of least privilege.

📋 Audit & Compliance

Enable Kafka authorizer logging. Ship logs to SIEM. Monitor with Prometheus + Grafana. Alert on unauthorized access.

Day 4 Complete!

Advanced Kafka Operations — Level 3

- ✓ Kafka Internals (Replica Manager, Group Coordinator, Controller)
- ✓ Performance Tuning (Broker, Producer, Consumer, OS)
- ✓ Cluster Scaling (Add Brokers, Rebalancing, Kafka Connect)
- ✓ Security (TLS/mTLS, Kerberos, SASL/SCRAM)

Next: Day 5 — HA/DR, Disaster Recovery & Real-World Scenarios